

Vesta – Functionality Test Scope & Feature Overview

Version: 3.0

Date: February 2026

Platform: Android (Flutter/Dart client, Python backend)

Backend API: <https://backend-vesta.onrender.com>

Deep Link Domain: <https://vestaapp.co>

Test Credentials

Field	Value
Username	JOIN
Password	Uu1111
Test Phone 1	+962 7 1234 5678
Test Phone 2	+962 7 1111 1111
OTP (test numbers)	000000

Note: These are Firebase test phone numbers. Real phone numbers receive actual SMS.

1. App Summary

Name: Vesta

Purpose: Personal Financial Management (PFM) app enabled by open banking.

Data Sources: JOPACC Open Banking sandbox (test banks & test customers) + Firebase (auth & app data).

Core Capabilities

1. Onboard and create an account with phone verification
 2. Link bank accounts via the JOPACC sandbox
 3. View consolidated balances and transactions
 4. Categorize spending
 5. Create a personal budget plan with customizable cycles
 6. Track spending vs budget with visual analytics
 7. Create a shared Household and track a joint budget
 8. Set and track savings goals
-

2. Onboarding & Authentication Flow

2.1 Sign Up (New User Journey)

Step 1: Fill Registration Form

Screen title: "Join Us Today"

Fields:

- **Username** – alphanumeric and underscores only (`^[a-zA-Z0-9_]+$`)
- **Email** – standard format (`^[\\w-\\.]+@([\\w-]+\\.){2,4}$`)
- **Password** – text field with show/hide toggle
- **Repeat Password** – text field with show/hide toggle

Password requirements are shown live below the password field as the user types, with a checkmark (green) or circle (grey) for each:

- At least 6 characters
- At least one uppercase letter (A–Z)
- At least one lowercase letter (a–z)
- At least one number (0–9)

Validation (in order):

1. All fields must be filled → "Please fill all fields"
2. Passwords must match → "Passwords do not match"
3. Password must meet all requirements → "Please ensure your password meets all requirements"
4. Username format check → "Username can only contain letters, numbers, and underscores"
5. Email format check → "Please enter a valid email address"

6. Username uniqueness check against `usernames` collection (fallback: `users` collection) → "Username already taken. Please choose another one."
7. Email uniqueness check against `emails` collection (fallback: `users` collection) → "Email already registered. Please use another email or login."

Button: **Continue** (navigates to phone number step if all checks pass)

Step 2: Phone Number Entry

Screen title: "Enter your phone number"

- Country code `+962` shown as fixed prefix
 - User enters Jordan number: 9 digits starting with 7 (`^7\d{8}$`)
 - Invalid format → "Enter a valid Jordan number starting with 7 (e.g. 79xxxxxxx)"
 - Button: **Send** → triggers Firebase phone verification (sends SMS)
 - On blocked/too-many-requests → shows "Phone Number Blocked" dialog: Block Duration: 30 minutes
-

Step 3: OTP Verification

Screen title: "Enter OTP"

- 6-digit input field
- Button: **Verify & Create Account**
- OTP validity: **2 minutes** (120 seconds), shown as live countdown "Code expires in MM:SS"
- Turns orange when ≤ 30 seconds remaining
- On expiry: "OTP has expired. Please request a new code."
- Resend cooldown: **60 seconds** — "Resend code in Xs" countdown
- After 60s: "Resend OTP" link appears
- Resending resets both the 2-minute validity timer and the 60-second cooldown
- Validation: must be exactly 6 digits → "Please enter a 6-digit OTP"
- On too many failed attempts: "Phone Number Blocked" dialog (30-minute block)

On successful OTP verification:

- Performs final uniqueness check for username and email
- Atomically creates (Firestore batch write):
- `usernames/{username.toLowerCase() }` document
- `emails/{email.toLowerCase() }` document
- `users/{uid}` document with fields:

```
username, email, phoneNumber,  
  
currency: "JOD",  
  
totalBalance: 0.0,  
  
totalIncome: 0.0,  
  
totalExpense: 0.0,  
  
dayOfMonth: 28,  
  
householdIds: [],  
  
createdAt: serverTimestamp
```

- Creates 4 default spending categories in `users/{uid}/categories/`:
 - Food And Drinks (expense)
 - Groceries (expense)
 - Entertainment (expense)
 - Others (expense)
 - Navigates to the Bank Linking screen (ChooseBankSplash)
-

2.2 Login (Returning User)

Screen title: "Welcome Back"

Subtitle: "Login with your username or email"

Fields:

- **Username or Email** – accepts either format
- **Password** – with show/hide toggle

Logic:

- If identifier contains @ → treats as email directly
- Otherwise → looks up username in `users` collection to get email, then authenticates

Buttons:

- **Login** – submits credentials
- **Fingerprint icon** – appears if device supports biometrics AND biometric login is enabled in Settings; grayed out if not enabled

Links:

- **Forgot Password?** → opens a dialog where user enters email → sends Firebase password reset email → "Password reset email sent. Please check your inbox."
- **Sign Up** → navigates to Register screen

Error messages:

- Wrong credentials → "Invalid username or password" (generic, no account enumeration)
- Too many attempts → "Too many attempts. Please try again later."
- Network error → "Network error. Please check your connection."

On successful login:

- `handleBudgetCycleOnLogin()` is called (background process — finalizes previous budget cycle, initializes current cycle if needed)
 - Navigates to Main Screen
-

2.3 Validation Rules Summary

Field	Rule	Error Message
Username	Alphanumeric + underscore only	"Username can only contain letters, numbers, and underscores"
Username	Must be unique	"Username already taken. Please choose another one."
Email	Valid email format	"Please enter a valid email address"
Email	Must be unique	"Email already registered. Please use another email or login."
Password	Must match repeat	"Passwords do not match"
Password	≥6 chars, uppercase, lowercase, number	"Please ensure your password meets all requirements"
Phone	Jordanian format (7XXXXXXXX)	"Enter a valid Jordan number starting with 7 (e.g. 79xxxxxx)"

Field	Rule	Error Message
OTP	Exactly 6 digits	"Please enter a 6-digit OTP"

3. Home Screen

Accessed after login. Shows:

- **App bar:** "Welcome {username}"
- **Side drawer** (hamburger menu): navigation to all main sections

Total Balance Card

- Displays JOD {totalBalance} (sum of all linked active accounts)
- Tapping navigates to the Accounts page

Income & Expense Row

- **Total Income** (green, arrow-up icon) – tappable → opens income input dialog to update income
- **Total Expense** (red, arrow-down icon) – read-only display

Quick-Action Buttons (2×2 grid)

Button	Destination
Budgeting	Personal Budget Screen
Spending s	Spending Analysis Screen
Savings	Savings Goals Screen
Househol d	Household List Screen

4. Accounts & Transactions

4.1 Accounts Screen

Accessible from: Profile → Linked Accounts, or Home → Total Balance card

Displays all accounts where `linked: true`.

Each account card shows:

- Bank name (e.g., "Bank of JoPACC")
- Account type (savings / current / salary / checking / payroll / merchant / operating)
- Masked IBAN — first 6 characters + `...` + last 4 digits (e.g., J094AB...3234)
- Balance (JOD)
- Status flags: active / suspended / closed; credit locked; debit locked (shown when applicable)

FAB (+): navigates to bank linking flow

4.2 Bank Account Linking (JOPACC Sandbox)

From Accounts screen, tap FAB → **ChooseBankSplash** → select **Bank of JoPACC**

JopaccLinkScreen – 3-step process:

Step 1: Login

- Enter JoPACC sandbox username and password
- Credentials stored in `providers.jopacc.username` field

Step 2: Consent

- User reviews and accepts data access permissions:
- Read account details and balances
- Read transactions
- Read standing orders

Step 3: Account Selection

- Backend fetches accounts from JOPACC API
- User selects which accounts to link (checkboxes)
- Linked accounts stored in `users/{uid}/accounts` with `linked: true`

Backend endpoint: `GET /sync_accounts/{uid}/{username}`

4.3 Transaction Sync

- User taps "Sync" from Accounts or Transactions screen
- App calls `GET /get_transactions/{uid}/{accountId}` for each linked account
- Backend fetches from JOPACC API and stores in Firestore

- Idempotent: re-syncing updates fields but **preserves user-assigned categories**
-

4.4 Standing Orders / Upcoming Payments Sync

- App calls GET `/get_sosps/{uid}/{accountId}` for each linked account
 - Standing orders stored under account sub-collection
 - Data includes: beneficiary, amount, currency, next payment date, status, frequency
-

4.5 Transactions Screen

Main View:

- All transactions from all linked accounts
- Sorted by date, newest first

Filters (chips/dropdowns):

- Account: "All Accounts" or specific account
- Category: "All Categories" or specific category
- Time period: All / Today / Last 7 Days / Last 30 Days / Month / Cycle

Transaction Card shows:

- Amount: green for credit (incoming), red for debit (outgoing), in JOD
- Masked account label (e.g., `**** 3234`)
- Date
- Merchant name
- Category dropdown (user can assign/change)
- **Assign button** — visible only if user is a member of at least one household; opens bottom sheet to select which household to assign the transaction to

Tap on transaction → full details:

- Full description
- Merchant information
- Transaction ID
- Account details
- Source: Open Banking / SMS / Manual

Transaction Sources:

1. openBanking — synced from JOPACC API

2. `sms` — user pastes a bank SMS, app parses amount, merchant, date, type
 3. `manual` — user manually creates transaction
-

4.6 Add Transaction (Manual Entry)

Access: "+" button on Transactions screen

Fields:

- **Amount** — numeric (must be > 0)
- **Type** — Debit or Credit toggle
- **Merchant** — text input
- **Notes** — text area
- **Date** — date picker (past dates only; `lastDate: DateTime.now()`)
- **Account** — dropdown of linked accounts (required)
- **Category** — category dropdown

Validation:

- Amount must be > 0
 - Account must be selected
 - Date cannot be in the future
-

5. Budgeting & Spending Analysis

5.1 Plan Budget (Setup Screen)

Access: from Personal Budget Screen

Purpose: Define monthly budget using percentage allocations.

Fields:

- **Total Monthly Income** (JOD) — note: this is planned/expected income
- **Savings %** — portion for savings goals
- **Essential Spending %** — necessities (rent, groceries, bills, food)
- **Luxuries & Entertainment %** — non-essential spending

Validation:

- Each field: 0–100%

- Sum of all three cannot exceed 100%
- Sum < 100% is allowed (unallocated buffer)

On save: Stores budget document at `users/{uid}/budget/{YYYY-MM}` and updates `users/{uid}.totalIncome`

5.2 Personal Budget Screen

Spending Budget Bar (top of screen):

- Top value: current cycle spending (JOD) = essential + luxury transactions
- Bottom value: budget amount = (income × spending%) + (income × luxuries%)
- Progress bar color: green (< 80%), yellow (80–100%), red (> 100%)
- Only counts transactions with assigned categories
- Only counts debit transactions within current cycle dates

6-Month Line Chart:

- Blue solid line: total spending per cycle
- Grey dashed line: budget plan per cycle
- Green solid line: actual savings transfers
- X-axis: last 6 budget cycles; Y-axis: JOD amounts

Budget Plan Card:

- Savings allocation: income × saving%
- Essential Spending allocation: income × spending%
- Luxuries allocation: income × luxuries%

Upcoming Payments:

- Lists active standing orders (SOSPs) showing:
- Beneficiary name
- Next payment date
- Amount (JOD)
- Frequency (e.g., Monthly, Weekly)
- Status indicator (active/inactive)

Spending Categories:

- Top 5 categories by absolute amount
- Each row: category icon & name, transaction count (current cycle), net amount (red for debit, green for credit)
- "See All" → full category breakdown

Latest Transactions:

- 5 most recent transactions (same card format as Transactions screen)
- Changing category on a transaction immediately recalculates spending bar

Time Period Filters:

- Day (today only)
 - Week (last 7 days)
 - Month (current calendar month)
 - Year (current calendar year)
 - Cycle (current budget cycle — default)
-

5.3 Budget Cycle Logic

The app uses a **customizable budget cycle** defined by a reset day (default: 28th).

Stored in: `users/{uid}.dayOfMonth` (default: 28)

Cycle calculation:

- If today's day $\geq$ reset day: cycle runs from {current month, reset day} $\rightarrow$ {next month, reset day - 1}
- Example (reset = 28): Jan 28 $\rightarrow$ Feb 27
- If today's day $<$ reset day: cycle runs from {previous month, reset day} $\rightarrow$ {current month, reset day - 1}
- Example (reset = 28, today = Jan 15): Dec 28 $\rightarrow$ Jan 27

Label month: The month the cycle ends in (e.g., Dec 28 $\rightarrow$ Jan 27 = "2026-01")

Category-to-Bucket Mapping:

Category	Bucket	Counted in Spending
Groceries	Essential	Yes
Food And Drinks	Essential	Yes
Entertainment	Luxury	Yes
Others	Luxury	Yes
Savings	Savings	No (separate tracker)
(uncategorized)	—	No

6. Savings Goals

6.1 Savings Page

Header card:

- Total Savings (JOD) — from `users/{uid}.totalSavings`
- Available to Allocate: Total Savings minus sum of all `currentAmount` across goals

Goals list: ordered by creation date (newest first)

Each goal card shows:

- Emoji icon, goal title, duration
- Progress bar: current / target amount
- Percentage achieved
- Monthly savings amount
- Remaining amount (floored at 0)
- "Completed" badge when fully funded

Interactions:

- **Tap** → Allocate dialog
 - **Long press** → Delete confirmation dialog
 - **FAB (+)** → Create savings goal
-

6.2 Allocate Dialog

Shown when tapping a goal card.

Displays:

- Available balance (`totalSavings` – already allocated)
- Current amount in goal
- Remaining amount to reach target

Input: amount to allocate (JOD)

Behaviour:

- Amount capped to remaining (if user enters more, it allocates only what's needed with a warning)
- Cannot exceed available balance → "Insufficient available balance"
- Cannot allocate to a fully funded goal → "This goal is already fully funded"
- "Remove from goal" link (visible if current amount > 0) → opens deallocate dialog

Deallocate dialog:

- Shows current amount in goal
 - Input: amount to remove
 - "Remove all" shortcut button
 - Cannot remove more than current amount
-

6.3 Create Savings Goal

Step 1 – Choose Goal Type:

Preset options:

- New Car 🚗
- Vacation 🌴
- House 🏠
- Emergency Fund 🆘

Button: **"Add New Saving Goal"** → Custom Goal dialog

Custom Goal dialog:

- Goal Title (free text)
- Emoji picker: 16 options (💰 🎓 💍 🎮 📱 🖥️ 🎸 ⚽ 🎨 📚 🌂 ✈️ 🏥 🧡 🐶 📺)
- Color picker: 8 pastel color options

Step 2 – Goal Details:

Fields:

- Target Amount (JOD)
- Calculate by: **Duration** or **Monthly Amount** (toggle chips)
- Duration mode: enter months → monthly amount auto-calculated
- Monthly mode: enter monthly amount → duration auto-calculated
- Duration field (months) — displays human-readable format (e.g., "1 year and 3 months")
- Monthly Savings field (JOD)

Summary card shown when all fields are filled.

Validation:

- Target amount must be > 0
- Duration must be > 0
- Monthly amount must be > 0

Button: **Create Savings Goal** → saves to `users/{uid}/savings/` collection

7. Household Shared Budget

7.1 Household List Screen

Access: Home → Household button or side drawer

If no households: empty state with "Create Household" button

If households exist: list of cards

Each card shows:

- Household name
- Member count
- Tap → Household Detail page
- Long press → delete confirmation dialog (irreversible; removes from `users/{uid}.householdIds` and deletes `households/{householdId}`)

FAB (+) / "Create Household" button: opens Create Household flow

7.2 Create Household

Field: **Household Name** (required)

On submit (atomic batch write):

- Creates `households/{householdId}` document:

```
householdName: string,  
  
createdBy: uid,  
  
members: [uid],  
  
budget: 0,  
  
budgetResetDay: (user's dayOfMonth),
```

createdAt: serverTimestamp

- Adds household ID to `users/{uid}.householdIds` array

Navigates to Household Detail page.

7.3 Household Detail Screen

Header:

- Household name (tapping the edit icon opens rename dialog)
- Horizontal scrollable member avatars: profile picture (if set) or first letter of username in colored circle, with username below

Spending Overview Chart:

- 6-cycle line chart
- Blue solid: total household spending per cycle (essential + luxury)
- Grey dashed: household budget per cycle
- Data from `households/{householdId}/transactions`

Budget Tracker Bar:

- Shows current cycle spending vs household budget amount
- Tap → input dialog to set flat JOD budget amount (saved to `households/{householdId}.budget`)
- Color coding same as personal budget bar (green/yellow/red)

Spending Categories:

- Top 5 categories by net amount (scoped to household transactions only)
- Shows: icon, name, transaction count (current cycle), net amount
- "See All" → full breakdown

Latest Transactions:

- 5 most recent transactions assigned to this household
- Shows: merchant, amount (red/green), date, category, "Assigned by: {username}"
- Tap "All Transactions" → Transactions screen filtered by this household

App bar actions:

- **Invite Member** (person_add icon) → generates invite link → share dialog
-

7.4 Inviting Members

Flow:

1. Tap invite icon in household detail app bar
2. App creates invites/{inviteId} document:

```
householdId, inviterUid, inviterName, householdName,  
  
status: "pending", createdAt: serverTimestamp
```

1. Deep link generated: `https://vestaapp.co/join?inviteId={inviteId}`
2. System share dialog opens (share via WhatsApp, SMS, Email, etc.)
 - Message: "Join my household '{householdName}' on Vesta! Click here: {inviteLink}"

Receiver flow – not logged in:

1. Taps deep link → app opens
2. Link stored internally in `_pendingInviteLink`
3. User goes through login/signup
4. After login: "You're Invited!" dialog appears automatically

Receiver flow – already logged in:

1. Taps deep link → app opens
2. "You're Invited!" dialog appears immediately

Invite dialog:

- Title: "You're Invited!"
- Message: "{InviterName} has invited you to join {HouseholdName}"
- Buttons: **Decline** | **Accept**

On Accept:

- Updates invite: `status: "accepted", acceptedByUid: currentUserUid`
- Cloud Function adds user to household members array and adds household to user's `householdIds`
- Success message: "Invite accepted! Joining household..."

Edge cases:

- Self-invite → "You can't accept your own invite."
- Invalid invite ID → "Invite link is invalid or expired."
- Invites do not expire (no expiration logic currently)

7.5 Assigning Transactions to a Household

From the Transactions screen, if user is a member of at least one household:

- Each transaction card shows an **Assign** button
 - Tap → bottom sheet lists user's households
 - Select household → transaction is copied to `households/{householdId}/transactions/{transactionId}` with `assignedBy` and `assignedAt` fields
 - Transaction now appears in the household's latest transactions, categories, and budget calculations
-

8. Profile Page

Avatar: Shows profile picture (if set) or initials (first letter of each name part). Small camera icon badge at bottom-right.

Tap avatar:

- Dialog: "Update Profile Picture"
- Options: "Choose from Gallery" or "Take a Photo"
- Image compressed to 50% quality, max 512×512px
- Uploaded to Firebase Storage at `profile_pictures/{uid}.jpg`
- Download URL saved to `users/{uid}.profileImageUrl`
- Loading spinner overlay shown during upload

Displayed info:

- Username
- Email
- Phone number
- "Member since {Month Year}"

Stats row:

- Linked Accounts count (only `linked: true`)
- Households count
- Savings Goals count

Menu items:

- **Linked Accounts** → Accounts screen

- **Budget Reset Day** → dialog to enter day of month (1–31)
- **Settings** → Settings screen

Logout:

- Confirmation dialog: "Are you sure you want to logout?"
 - On confirm: clears secure storage (tokens), signs out Firebase, navigates to login screen
-

9. Settings Screen

Security section:

- **Biometric Login** toggle (only shown if device supports biometrics)
 - Enabling: prompts biometric auth first to confirm ("Authenticate to enable biometric login")
 - If biometrics not available: shows "Not available on this device"
 - If enabled: fingerprint icon on login screen becomes active
-

10. Navigation Structure

Bottom navigation bar (MainScreen):

Tab	Screen
Home	Home page
Transactions	Transactions list
(center)	—
Spending	Spending Analysis
Profile	Profile page

Side drawer: accessible from Home screen hamburger menu

11. Deep Link Handling

Domain: vestaapp.co

Path: /join

Query parameter: inviteld

Handled by: DeepLinkService (singleton, initialized once in main app widget)

Scenarios:

- **Cold start (app terminated):** app opens → user is not logged in → link stored → shown after login
 - **Warm link (app running):** dialog shown immediately if user is logged in; stored for after-login if not
-

12. Budget Cycle Finalization (Automated)

Triggered automatically on every login via `handleBudgetCycleOnLogin()`.

Process:

1. Determine current and previous cycle dates based on `dayOfMonth`
2. If previous cycle budget document exists:
 - Query all transactions within previous cycle date range
 - Calculate actual spending per category-to-bucket mapping
 - Calculate actual savings
 - Update previous cycle document with `actualSpending` and `actualSavings`
1. If current cycle document doesn't exist:
 - Copy percentages from last known plan
 - Use current `totalIncome`
 - Create new cycle document with expected values and date timestamps

This is a background process — user sees no UI for it.

13. Known Limitations (Not Yet Implemented)

- No Arabic language support
- No in-app "Change Password" feature (forgot password via email works)
- Spending budget bar does not open a detailed view on tap
- Total savings amount cannot be manually edited by the user
- Invite links do not expire
- SMS transaction parsing has no format validation (any pasted text is attempted)

- Account label masking is inconsistent between Open Banking transactions (masked) and manually entered transactions (not masked)